**Ecole Nationale Supérieure d'Electricité et de Mécanique**

# Problème du voyageur de commerce

Didier Maquin

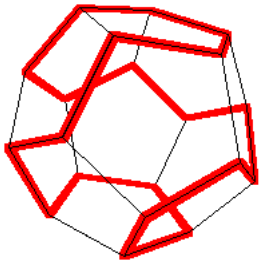
Note : tous les liens bleus sont cliquables

## 1 Enoncé du problème

Un voyageur de commerce (TSP : *Traveling Salesman Problem*) doit visiter $n$ villes données en passant par chaque ville exactement une fois. Il commence par une ville quelconque et termine en retournant à la ville de départ. Les distances entre les villes sont connues. Quel chemin faut-il choisir afin de minimiser la distance parcourue ?

## 2 Historique

$19^{ième}$ siècle — Les premières approches mathématiques exposées pour le problème du voyageur de commerce (PVC) ont été traitées au $19^{ième}$ siècle par les mathématiciens Sir William Rowan Hamilton et Thomas Penyngton Kirkman. Hamilton en a fait un jeu : *Hamilton's Icosian game*. Les joueurs devaient réaliser une tournée passant par 20 points en utilisant uniquement les connections prédéfinies.



| | |
|---|---|
| années 1930 | Le PVC est traité plus en profondeur par Karl Menger à Harvard. Il est ensuite développé à Princeton par les mathématiciens Hassler Whitney et Merril Flood. Une attention particulière est portée sur les connections par Menger et Whitney ainsi que sur la croissance du PVC. |
| 1954 | Solution du PVC pour 49 villes par George Dantzig, Delbert Fulkerson et Selmer Johnson par la méthode du cutting-plane[1]. |
| 1975 | Solution pour 67 villes par Camerini, Fratta and Maffioli. |
| 1987 | Solution pour 532, puis 2 392 villes par Padberg et Rinaldi. |
| 1998 | Solution pour les 13 509 villes des Etats-Unis. |
| 2001 | Solution pour les 15 112 villes d'Allemagne par Applegate, Bixby, Chvàtal et Cook des universités de Rice et Princeton[2]. |

1. G.B. Dantzig, R. Fulkerson, S.Johnson, *Solution of a large-scale traveling salesman problem*, Operations Research 2 : 393-410, 1954.
2. D. Applegate, R. Bixby, V. Chvàtal and W. Cook, *Implementing the Dantzig-Fulkerson-Johnson algorithm for large traveling salesman problems*, Mathematical Programming Series B, 97 :91–153, 2003.

## 3 Aspect formel

A partir d'une matrice $C = (C_{ij})$ où $C_{ij}$ représente les distances entre la ville $i$ et la ville $j$ $(1 \leq i, j \leq n)$, il faut trouver une permutation

$$\sigma = (\overset{1}{\sigma_1} \quad \overset{2}{\sigma_2} \quad \dots \quad \overset{n}{\sigma_n})$$

qui minimise la somme des distances $C_{\sigma(1)\sigma(2)} + C_{\sigma(2)\sigma(3)} + \cdots + C_{\sigma(n-1)\sigma(n)} + C_{\sigma(n)\sigma(1)}$. Autrement dit, il faut trouver un cycle hamiltonien de longueur minimale dans un graphe valué.

## 4 Complexité

Ce problème est un représentant de la classe des problèmes NP-complets [3]. L'existence d'un algorithme de complexité polynomiale reste inconnue. Un calcul rapide de la complexité montre qu'elle est en $O(n!)$ où $n$ est le nombre de sommets. Le tableau 1 montre l'explosion combinatoire du PVC pour un graphe complet.

| # sommets | # arcs | # possibilités |
|---|---|---|
| 5 | 10 | 12 |
| 6 | 15 | 60 |
| 7 | 21 | 360 |
| 8 | 28 | 2 520 |
| 9 | 36 | 20 160 |
| 10 | 45 | 181 440 |
| 15 | 105 | $43 \times 10^9$ |
| 20 | 190 | $6 \times 10^{16}$ |
| 25 | 300 | $31 \times 10^{22}$ |

Table 1 – Nombre de possibilités de chemins en fonction du nombre de villes

S'il faut 1 ns pour examiner une possibilité, avec un graphe possédant 100 sommets, il faut $10^{158}$ ns soit $10^{132}$ fois l'âge de l'univers (estimé à 13 milliards d'années) !

## 5 Intérêt

Le PVC fournit un exemple d'étude d'un problème NP-complet dont les méthodes de résolution peuvent s'appliquer à d'autres problèmes de mathématiques discrètes. Néanmoins, il a aussi des applications directes, notamment dans les transports et la logistique. Par exemple, trouver le chemin le plus court pour les bus de ramassage scolaire ou, dans l'industrie, pour trouver la plus courte distance que devra parcourir le bras mécanique d'une machine pour percer les trous d'un circuit imprimé (les trous représentent les villes).

3. Voir annexe

# 6 Méthodes de résolution du problème

Les algorithmes pour résoudre le PVC peuvent être répartis en deux classes :

- les algorithmes déterministes qui trouvent la solution optimale ; leur complexité est exponentielle, ils sont très gourmands en puissance de calcul. Les algorithmes les plus efficaces sont basés sur la méthode des plants sécants, *cutting-plane* introduite par Ralph Gomory qui s'appuie sur une formulation en termes d'optimisation linéaire.
- les algorithmes d'approximation qui fournissent une solution sous-optimale. Ils permettent de trouver une solution dont le coût est proche du coût de la solution optimale. Ils ont l'avantage de permettre de trouver une solution en un temps raisonnable .

# 7 Quelques méthodes sous-optimales de résolution du PVC

## 7.1 Méthode par insertion

Une famille de méthodes est celle des algorithmes gloutons [4], dont l'un des plus connus est la méthode par insertion. On détermine un cycle initial passant par un sous-ensemble donné de sommets. On modifie ensuite ce cycle en insérant progressivement un à un les sommets restants jusqu'à ce que tous les sommets soient insérés. La mise en œuvre de cette méthode nécessite donc de répondre aux questions : comment choisir le cycle initial, dans quel ordre insérer progressivement les autres sommets et où insérer ce sommet ?

Le cycle initial peut être choisi comme l'enveloppe convexe (*convex hull*) des différents sommets. De nombreuses stratégies d'insertion peuvent alors être choisies :

- le plus proche (respectivement le plus éloigné) : il s'agit d'insérer le sommet dont la distance minimale à un sommet du cycle est minimale (respectivement maximale) ;
- coût minimal (respectivement maximal) : on insère le sommet qui fera le moins (respectivement le plus) augmenter la longueur du cycle.

Applet java de démonstration

## 7.2 Les plus proches voisins

Cette méthode fait également partie des algorithmes gloutons. Partant d'un sommet, on se rend au sommet le plus proche non encore visité et on réitère le processus. Une fois qu'on a visité tous les sommets du graphe, on retourne au premier pour fermer le cycle. Cette méthode fournit un résultat en un temps très bref (complexité en $O(n^2)$). Qualitativement, on voit (figure 1) que le résultat est améliorable. Il comporte des croisements (qu'il serait avantageux d'éliminer) et lorsque l'on s'approche de la fin, l'algorithme doit relier des sommets épars qui ont été négligés auparavant et qui sont très éloignés. La pénalité pour ne pas s'en être occupé auparavant est relativement importante, comme on le voit sur la figure 1.

Applet java de démonstration

4. Un algorithme est dit glouton (*greedy* en anglais) lorsqu'il fait le choix de l'optimum local à chaque étape.

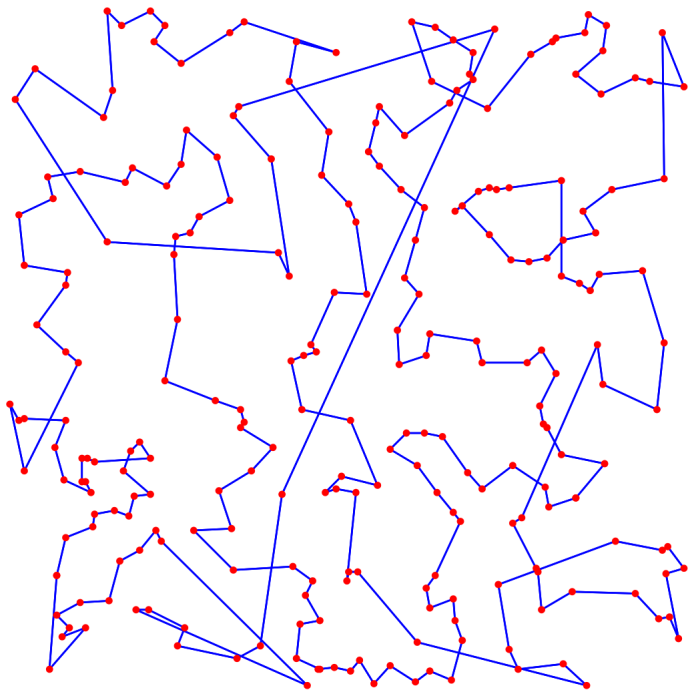


FIGURE 1 – Cycle obtenu par la méthode des plus proches voisins

### 7.3 L'arbre couvrant de poids minimal

Si les longueurs des arêtes respectent l'inégalité triangulaire, on peut construire un arbre couvrant de poids minimal grâce à l'algorithme de Prim. Selon son implémentation, la complexité de l'algorithme varie. La meilleure implémentation (utilisant des tas binaires) a une complexité en $O(n^2 \log n.)$. Effectuons maintenant un parcours en profondeur, *depth first search* de l'arbre comme sur la figure 2.

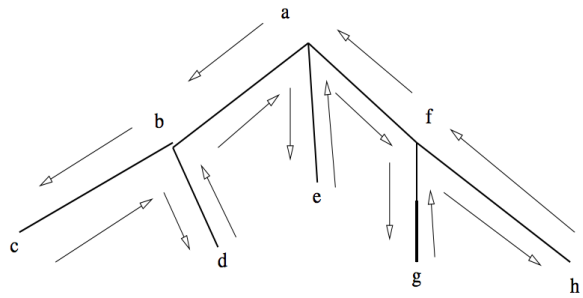



FIGURE 2 – Parcours en profondeur d'abord

Si l'on n'utilise pas de marquage des sommets, un sommet ayant $k$ fils apparaîtra $k+1$ fois dans le parcours. Pour l'exemple considéré, le parcours est abcbdbaeeafgfha. Chaque arête est ainsi parcourue deux fois : une fois dans chaque sens. Ainsi, la longueur du parcours est exactement deux fois le poids de l'arbre couvrant. Maintenant, supprimons les sommets apparaissant plus d'une fois, en conservant seulement leur première occurrence dans le parcours, puis en supprimant les suivantes. Le parcours donné en exemple devient :

abcdefgh. Si l'on ajoute une occurrence du premier sommet à la fin, on obtient un cycle hamiltonien. En procédant ainsi, la longueur du cycle ne peut être qu'inférieure à deux fois le poids de l'arbre couvrant. En effet, l'inégalité triangulaire garantit que la longueur du parcours n'augmente pas lorsqu'on retire les sommets récurrents.

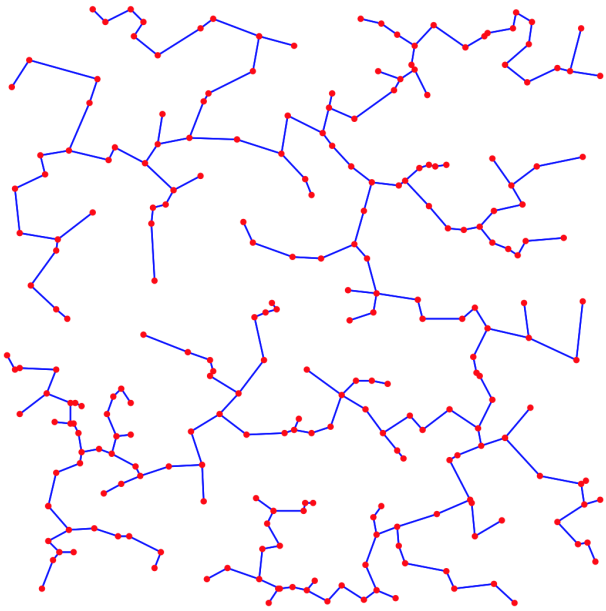


FIGURE 3 – Arbre couvrant de poids minimal

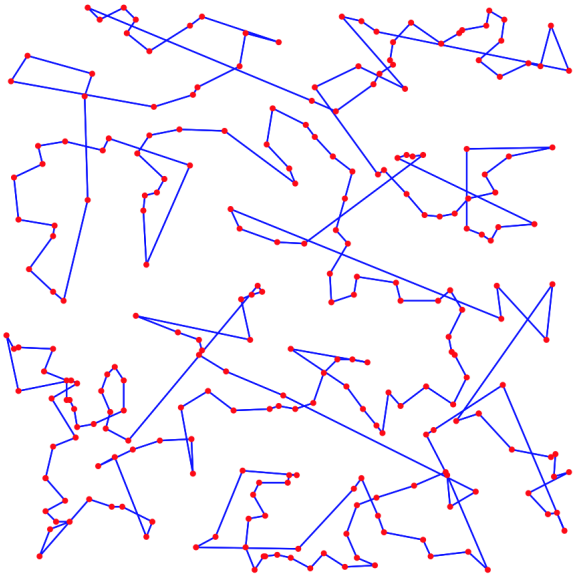


FIGURE 4 – Parcours issu d'un arbre couvrant de poids minimal

### 7.4 Optimisation locale

Les solutions fournies par les méthodes précédentes sont peu satisfaisantes car elles comportent des croisements ou des arêtes de grande longueur. Afin d'améliorer les solutions obtenues, on peut mettre en œuvre une optimisation locale. Le principe de l'heuristique 2-opt est d'examiner chaque paire d'arêtes non consécutives. Soit $a$ l'arête liant les sommets $A$ et $B$ et $b$ l'arête liant les sommets $C$ et $D$ dans le cycle initial. Si $\text{dist}(A, B)+\text{dist}(C, D) > \text{dist}(A, D)+\text{dist}(B, C)$, on crée un nouveau cycle en supprimant $a$ et $b$ et en insérant deux nouvelles arêtes, l'arête $c$ entre $A$ et $D$ et l'arête $d$ entre $C$ et $B$.

Considérons par exemple le parcours de gauche ci-dessous (représentant une partie de cycle hamiltonien) constitué de la succession des sommets 1–2–3–4–5–6–7–8–9. Si l'on effectue la "permutation" des arêtes 2–3 et 7–8 (décroissement des arêtes dans ce cas), on obtient le parcours représenté à droite 1–2–7–6–5–4–3–8–9.

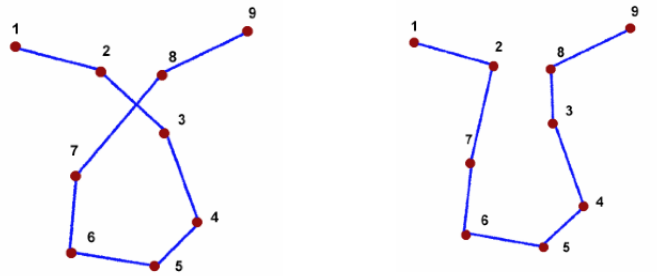



FIGURE 5 – Illustration de l'heuristique 2-opt

On notera que le second parcours s'obtient aisément. Ici, parmi les quatre sommets impliqués (2, 3, 7 et 8) le sommet 2 est le premier dans la séquence décrivant le cycle hamiltonien initial et le sommet 7 le dernier. Le nouveau parcours se déduit aisément du précédent en inversant l'ordre des sommets situés entre ce premier et ce dernier sommet (1–2–7–6–5–4–3–8–9).

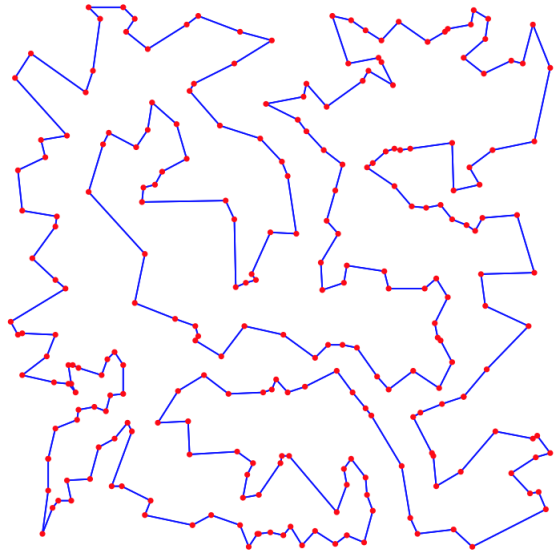


FIGURE 6 – Parcours après optimisation 2-opt

Applet java de démonstration

Ce document n'est pas original. De nombreux emprunts ont été effectués aux documents cités en référence.

## Références


[1] http ://sis.univ-tln.fr/~tollari/TER/AlgoGen1/node4.html

[2] http ://www-e.uni-magdeburg.de/mertens/TSP/TSP.html

[3] Charles Bouillaget. Le problème du voyageur de commerce, juin 1984.

[4] http ://www.tsp.gatech.edu/

[5] Christian Vassard. Le problème du voyageur de commerce. *In* Maths en scène. 22 articles pour se repérer dans l'exposition maths 2000. Brochure de l'Association des Professeurs de Mathématiques de l'Enseignement Public n° 121, pp. 179-189, Paris 1998.

# Annexe : Algorithme, efficacité, décidabilité algorithmique

## Notion d'algorithme

La notion d'algorithme, bien qu'ancienne, n'a été définie et étudiée qu'au XXe siècle, quand il s'est agi pour les mathématiciens de réfléchir aux fondements logiques des mathématiques. Essayons en quelques mots de préciser ce que cette notion recouvre.

**Un algorithme peut être défini comme un processus, une suite d'instructions permettant la résolution d'un problème donné dans toute sa généralité et en particulier quelle que soient les données du problème**.

Plus précisément, un algorithme doit vérifier les conditions suivantes :

1. il doit pouvoir être écrit dans un certain langage (c'est-à-dire un ensemble de mots écrits dans un alphabet défini) ;
2. des données lui sont soumises au départ (ce sont des entrées) ;
3. l'algorithme lui-même est un processus s'exécutant étape après étape ;
4. l'action à chaque étape est strictement déterminée par l'algorithme, les entrées et les résultats obtenus dans les étapes précédentes (on notera le caractère strictement déterministe de cette action) ;
5. l'algorithme retourne une réponse clairement spécifiée (de préférence celle que l'on attend...), appelée sortie ;
6. quelles que soient les entrées, l'exécution doit se terminer en un nombre fini d'étapes.

C'est un concept-clé de l'informatique : l'ordinateur ne peut traiter que les problèmes pour lesquels un algorithme existe. Cette définition pose deux questions auxquelles nous tenterons d'apporter une réponse dans la suite :

- existe-t-il un algorithme pour n'importe quel problème donné ?
- un algorithme doit-il finir dans un intervalle de temps raisonnable et qu'est-ce d'ailleurs qu'un intervalle de temps raisonnable ?

## Efficacité des algorithmes

**Problèmes indécidables** : Une question a intrigué les mathématiciens : existe-t-il un algorithme pour n'importe quel problème donné ?

La réponse à cette question est non : dans les années 30, plusieurs mathématiciens (dont Kurt Gödel, Alonzo Church, Alan Turing et Emil Post) ont montré l'existence de problèmes qui ne peuvent pas être résolus par des algorithmes (c'est-à-dire pour lesquels il n'existe aucune méthode de résolution) [5]. Ces problèmes furent qualifiés d'indécidables et, en quelque sorte, ils marquent l'horizon de ce qu'un mathématicien peut traiter.

5. C'est le cas du $10^{ème}$ problème de Hilbert. Un polynôme $P(x, y, z, ...)$ est dit diophantien si et seulement si tous ses coefficients sont dans $\mathbb{Z}$. Une équation diophantienne est une équation de la forme $P(x, y, z, ...) = 0$ où $P$ est un polynôme diophantien. La formulation du $10^{ème}$ problème de Hilbert est la suivante : existe-t-il un algorithme général permettant de déterminer si une équation diophantienne quelconque $P(x, y, z, ...) = 0$ admet une solution en nombres entiers ? Le problème a été résolu par Yuri Matiiassevitch en 1970 et la réponse est négative.

Déclarer un problème indécidable, c'est à la fois une grande victoire de l'esprit humain (une telle démonstration est souvent une prouesse mathématique) mais aussi, semble-t-il, un constat d'échec (c'est le glas de toute recherche sur ce problème). Ce dernier point est à relativiser car, si le problème dans sa généralité est définitivement clos, il peut cependant être résolu dans certains cas particuliers.

**Efficacité des algorithmes** : Un algorithme doit-il finir dans un intervalle de temps raisonnable ?

D'un point de vue théorique, avoir un nombre fini d'étapes est la seule contrainte que l'on impose à un algorithme ; mais d'un point de vue pratique, la réponse à un problème donné doit pouvoir être obtenue à échelle humaine : à ce titre, l'existence d'un algorithme ne suffit pas à ce que le problème soit concrètement résolu. L'efficacité des algorithmes a été initialement étudiée vers le milieu des années 60 par A. Cobham et J. Edwards. Pour évacuer le facteur temps, par trop subjectif, on fait intervenir le nombre d'étapes nécessaires pour mener l'algorithme à son terme. Un algorithme est dit à complexité polynomiale s'il existe deux entiers fixes $A$ et $k$ tels qu'à partir des données de longueur $n$, l'algorithme demande au plus $An^k$ étapes.

Ainsi, la multiplication de deux entiers est à complexité polynomiale. Quand on multiplie deux entiers de $n$ chiffres chacun, on effectue $n^2$ multiplications de deux entiers à un chiffre.

Les algorithmes qui ne sont pas à complexité polynomiale sont dits à complexité exponentielle. Ainsi un algorithme qui requiert $2^n$, $n^n$ ou $n!$ étapes pour des données de longueur $n$ est à complexité exponentielle : la fonction qui donne le nombre d'étapes n'est pas obligatoirement une fonction exponentielle au sens usuel.

L'algorithme exhaustif du problème du voyageur de commerce est un algorithme à complexité exponentielle : cette complexité résulte de l'explosion combinatoire que l'on observe quand on augmente le nombre de villes. Par définition, les algorithmes efficaces sont les algorithmes de complexité polynomiale ; les algorithmes inefficaces sont les algorithmes de complexité exponentielle.

**Classification des différents types de problèmes**

L'approche précédente permet d'amorcer une classification des types de problèmes que peut rencontrer un mathématicien. Tout d'abord, il existe des problèmes indécidables (appelés aussi problèmes sans preuve), c'est-à-dire pour lesquels il n'existe pas d'algorithme. Les autres problèmes sont donc ceux pour lesquels existe un algorithme : un problème est dit de type P s'il existe pour ce problème un algorithme de résolution de complexité polynomiale (c'est le cas du problème du calcul du PGCD de deux entiers, résolu par l'algorithme d'Euclide).

Si tous les algorithmes de résolution sont de type exponentiel, le problème est dit à preuve difficile (vous comprendrez que la difficulté de cette définition réside dans le "tous" !). Le problème du voyageur de commerce est-il un problème à preuve difficile ? La réponse est loin d'être simple. Le seul algorithme que l'on connaisse, l'algorithme exhaustif, est de type

exponentiel et l'on ne connaît pas à ce jour d'algorithme polynomial. Mais, pour autant, personne n'a démontré qu'il n'en existait pas. C'est bien là toute la difficulté des problèmes à preuve difficile ! En toute rigueur, le problème du voyageur de commerce ne peut pas être considéré comme un problème à preuve difficile : il a cependant toutes les chances de l'être.

Il existe une autre catégorie de problèmes, englobant la classe P : c'est la classe NP (*non deterministic polynomial problem*). Le problème du voyageur de commerce en est le prototype. Comment décrire la classe NP ? Outre les problèmes de la classe P, elle regroupe des problèmes tous solubles (et donc pas indécidables) mais de type incertain : un algorithme inefficace de résolution existe et l'on ne sait pas s'il est possible de trouver un algorithme efficace pour ces problèmes. Une autre caractéristique de ces problèmes réside dans le fait que si l'on dispose d'une solution, on peut vérifier si elle satisfait le problème par un algorithme efficace (de complexité polynomiale).

L'intérêt de la classe NP réside donc en deux points. Le premier est que de nombreux problèmes pour lesquels on n'a pas encore trouvé d'algorithme efficace se révèlent être de type NP. Citons un exemple, la factorisation d'un entier. C'est un problème pour lequel le seul algorithme connu est inefficace : c'est l'algorithme de recherche exhaustive de tous les facteurs premiers possibles. Pour de petits entiers, la réponse est donnée rapidement ($21 = 3 \times 7$) ; pour des entiers plus importants, les temps de calcul croissent de façon exponentielle (essayez pour voir de factoriser le nombre entier 86859557862374317150965493077 9). Peut-être existe-t-il un algorithme efficace ; nul ne l'a encore trouvé. Ce problème est lui aussi de classe NP. La solution est : $9876543219876543251 \times 796695301529$. Si vous n'êtes pas convaincu, faites la multiplication : cela ne vous prendra qu'un temps polynomial !

Le second point, c'est que Stephen Cook a montré en 1971 qu'il était hautement improbable que l'on trouve pour certains problèmes NP (dits NP-complets) un algorithme efficace : en ce sens que si l'on trouve un algorithme efficace pour un tel problème, on en déduira un algorithme efficace pour n'importe quel problème NP. Or, croire que l'on résoudra le cas de tous les problèmes NP semble excessivement optimiste, donc on pense qu'on ne parviendra pas à résoudre le cas d'un problème NP-complet. Ceci étant, le problème demeure ouvert exprimé dans la littérature par la formule $P \stackrel{?}{=} NP$.

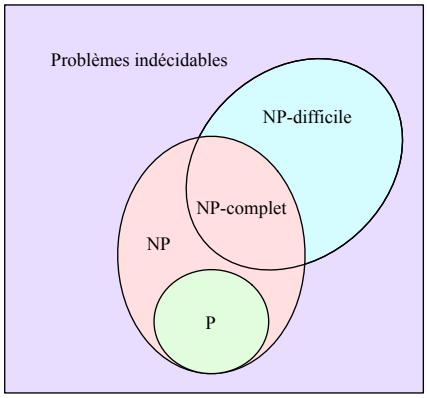



FIGURE 7 – Classification des problèmes (sous l'hypothèse $P \neq NP$)

La figure 7 résume la classification des problèmes :

- P est l'ensemble de problèmes de décision algorithmique dont la complexité est polynomiale ;
- NP est l'ensemble de problèmes de décision algorithmiques tels que, si une solution est donnée, il est possible de vérifier cette solution en un temps polynomial. On a donc $P \subset NP$ ;
- la classe des problèmes NP-complet rassemble les problèmes de NP qui sont aussi difficiles que tout problème de NP. Ces problèmes sont dits faiblement équivalents [6] entre eux. Cette catégorie est d'une importance particulière car si quelqu'un réussit à prouver qu'un seul problème de la classe NP-complet est effectivement dans P, alors tout NP est inclus dans P !
- la classe des problèmes à preuve difficile (aussi appelé NP-difficile, ce qui est ambiguë), rassemble les problèmes au moins aussi difficile que tout problème de NP. Les problèmes NP-complets et NP-difficiles sont souvent confondus. La différence est qu'un problème NP-difficile n'a pas besoin d'être dans NP (par exemple parce que c'est un problème si difficile que même le tester sur une seule solution ne peut être accompli en un temps polynomial)

6. On dit qu'un problème $Q$ est *faiblement réductible* à un problème $P$ ($Q \propto P$) s'il existe une transformation $f$ qui, pour toute instance $q$ du problème $Q$, construit en un temps polynomial en la taille de $q$, une instance $p = f(q)$ du problème $P$ de taille polynomiale de la la taille de $q$ et tel que l'instance de $Q$ a une solution si et seulement si l'instance de $P$ a une solution. Deux problèmes $P$ et $Q$ sont dits *faiblement équivalents* si $P$ est réductible à $Q$ et $Q$ réductible à $P$.